Evolução da Arquitetura DataOps Geoespacial: Otimização de Renderização WebGL, Modelagem Snowflake e Governança Orientada a IA

Gabriel Gonçalves Sampaio[1], **Barbara Pimenta Caetano**[1]

[1]Instituto de Engenharia de Sistemas e Tecnologia da Informação (IESTI)
Universidade Federal de Itajubá (UNIFEI)
Avenida BPS, 1303 – 37500-903 – Itajubá – MG – Brasil

gabrielsampaio059@gmail.com, bpimentacaetano@unifei.edu.br

Abstract. *The growing demand for real-time geospatial data processing poses challenges related to the ingestion, storage, and continuous analysis of large volumes of information. This paper details the evolution of an event-driven DataOps architecture previously established. The current iteration focuses on three fundamental pillars: extreme optimization of vector payloads for WebGL rendering (transitioning from WKT to Path JSON arrays), strict enforcement of the Snowflake modeling in the streaming fact table to reduce Apache Druid's memory footprint, and the implementation of declarative DataOps governance. This governance includes the transition to Software-Defined Assets in Apache Dagster and the establishment of rigid architectural guidelines for AI-assisted development.*

Resumo. *A crescente demanda por processamento de dados geoespaciais em tempo real impõe desafios na ingestão, armazenamento e análise contínua de grandes volumes de informações. Este trabalho detalha a evolução de uma arquitetura DataOps orientada a eventos previamente estabelecida. A iteração atual foca em três pilares fundamentais: a otimização extrema de payloads vetoriais para renderização WebGL (transição de WKT para arrays Path JSON), a adoção estrita da modelagem Snowflake na tabela fato de streaming para redução do consumo de memória no Apache Druid, e a implementação de governança DataOps declarativa. Essa governança abrange a transição para Software-Defined Assets no Apache Dagster e o estabelecimento de manuais arquiteturais rígidos para o desenvolvimento assistido por Inteligência Artificial.*

1. Introdução e Nova Intenção do Projeto

O mercado atual exige que informações geoespaciais sejam processadas e entregues com latências mínimas. Em nosso trabalho anterior, apresentamos uma arquitetura DataOps orientada a eventos capaz de realizar a ingestão contínua, o cálculo de rotas espaciais e a clusterização de anomalias térmicas (focos de incêndio) advindas do INPE. Essa plataforma provou a viabilidade técnica da integração entre Apache Kafka, o banco colunar Apache Druid e o motor de renderização Apache Superset em ambientes de recursos computacionais limitados.

No entanto, o avanço do projeto revelou que a simples ingestão funcional não era suficiente para garantir estabilidade a longo prazo. Gargalos arquiteturais foram identificados na forma como as geometrias pesadas eram trafegadas para o *frontend* e em como

atributos redundantes sobrecarregavam a memória RAM durante a ingestão massiva no Apache Druid.

A nova intenção do projeto não é apenas "fazer os dados fluírem", mas consolidar a plataforma como um modelo de referência de classe *Enterprise*. O foco agora recai sobre a otimização algorítmica, o banimento de dados textuais na esteira de *streaming*, a orquestração declarativa impecável e o controle restrito de código por meio de Inteligências Artificiais atuando sob manuais de governança.

2. Mudanças Implementadas e Embasamento Teórico

2.1. Otimização Vetorial: Transição do WKT para o *Path JSON*

Na versão original da arquitetura, o banco analítico armazenava e trafegava geometrias complexas (como polígonos de terras indígenas e infraestruturas elétricas lineares) utilizando o formato padronizado WKT (*Well-Known Text*).

A teoria subjacente à renderização geoespacial web moderna, fundamentada no WebGL (como a biblioteca Deck.gl operante no Apache Superset), expõe uma fraqueza no uso de WKT. Enviar dezenas de megabytes de coordenadas literais em *strings* forçava a *thread* principal do JavaScript a realizar *parsers* massivos e computacionalmente caros utilizando expressões regulares antes que as matrizes de dados chegassem à GPU. Isso gerava latência e quedas drásticas de quadros por segundo (*framerate*).

Nesta evolução, toda a camada de processamento de dimensões (ETL) foi reescrita para expurgar o WKT. O formato adotado é o *Path JSON*, consistindo em matrizes numéricas multidimensionais diretas (e.g., `[[[lon, lat], ...]]`). O embasamento teórico reside no fato de que arrays JSON aninhados são decodificados quase nativamente e passados para estruturas do tipo *TypedArray* na camada WebGL, ignorando etapas de *parsing* onerosas e permitindo uma fluidez sem precedentes na renderização de malhas de alta densidade no Superset.

2.2. Desnormalização Radical: Enforcement do Modelo *Snowflake*

O *payload* de focos de incêndio original disparado assincronamente pelo produtor Python para o Apache Kafka carregava, por conveniência, diversas descrições em formato de *string* (e.g., nomes de estados, classificações de infraestruturas afetadas).

Tabelas fato excessivamente largas (*Wide Tables*), repletas de textos redundantes, impactam a performance do *broker* do Kafka e inflacionam linearmente a alocação de *heap memory* das tarefas de ingestão (*Peons/MiddleManagers*) do Apache Druid. O modelo dimensional Kimball estabelece que a tabela fato deve ser a mais delgada possível.

Para alinhar o projeto a essa base teórica de *Data Warehousing*, aplicou-se uma filtragem restritiva utilizando o validador *Pydantic*. A tabela fato foi limpa de todas as descrições literais. Atualmente, trafegam na fila de streaming exclusivamente Chaves Estrangeiras (*Foreign Keys*) numéricas/codificadas e métricas matemáticas precisas (como distâncias euclidianas e índices de severidade ambiental). O Apache Druid assume o fardo da junção textual através da funcionalidade de *Lookups* dinâmicos em tempo de consulta, promovendo compactação máxima de *bytes* na rede.

2.3. Governança Declarativa: *BaseETL* e *Software-Defined Assets*

Em sistemas DataOps em rápida expansão, a reprodutibilidade é a métrica principal de qualidade. Inicialmente, as rotinas de carga (ETL) residiam em scripts independentes engatilhados por agendadores de tarefas tradicionais do Sistema Operacional (CRON).

A teoria do DataOps declarativo prescreve a observabilidade sistêmica. Diante disso:

1. Todos os *pipelines* batch de ingestão territorial foram reescritos sob o paradigma de Programação Orientada a Objetos, herdando de uma classe abstrata unificadora (`BaseETL`), o que concentra o controle geográfico (CRSs) e a geração do *Path JSON* numa única interface.
2. A orquestração foi migrada integralmente para o Apache Dagster. Scripts paralelos foram transformados em *Software-Defined Assets* (@asset). Essa abordagem gera um Grafo Acíclico Dirigido (DAG) tipado, permitindo visualizar a linhagem dos dados (*Data Lineage*), monitorar falhas inter-dependentes e assegurar agendamento reproduzível.
   Do ponto de vista prático e experimental, a arquitetura orquestrou estritamente os 7 principais *pipelines* modulares (Terras Indígenas, Municípios, Hospitais, Rodovias, Subestações, Sistemas Isolados e Linhas de Transmissão). Testes de execução validaram que essa migração centralizou a lógica em um ponto focal invariável, eliminou a concorrência problemática entre CRONs do Sistema Operacional, e possibilitou a validação de cabeçalhos e coerência das matrizes espaciais antes da sua injeção nas Tabelas de Dimensões.

2.4. Governança de IA e Engenharia de Qualidade

Na etapa inicial do projeto, a geração de rotinas e blocos lógicos dependia de interações externas via *APIs* isoladas ou instâncias de Inteligência Artificial abertas diretamente em navegadores web. Esta dinâmica engessava a rastreabilidade e limitava o escopo de atuação do algoritmo gerativo.

A evolução do projeto marcou a migração para Agentes de Desenvolvimento Autônomos diretamente embarcados no ambiente de codificação, dispondo de ferramentas nativas de leitura profunda de diretórios, manipulação de arquivos e execução de comandos *Bash*. Esse novo modelo hiper-acelerou as refatorações arquiteturais. No entanto, em uma topologia complexa sob a tutela direta de Inteligência Artificial, lacunas de contexto ocasionavam alucinações técnicas graves, como a regressão em métodos já refatorados, a poluição do *payload* Kafka, ou o acionamento indesejado do interpretador Python global em detrimento do ambiente virtual isolado.

Como contra-medida teórica, foi introduzida a figura dos Manuais Arquiteturais de Contexto (`agents.md`). Esses arquivos Markdown funcionam como *Guardrails* (barreiras de proteção) injetadas diretamente na base de conhecimento do agente de desenvolvimento. Eles preestabelecem "Leis de Ouro", exigindo, por exemplo:

- Execução obrigatória via binários contidos no *Virtual Environment* (`uv run`).
- Proibição formal do resgate do formato WKT nas camadas espaciais.
- Obrigatoriedade de criação de testes parametrizados utilizando a suíte `pytest` sempre que novas entidades dimensionais ou regras matemáticas entrarem em produção.

3. Conclusão

A modernização descrita eleva o protótipo DataOps a uma fundação estrutural altamente profissional. A adoção rigorosa do array *Path JSON* em detrimento do WKT resultou numa otimização imediata na renderização das *dashboards*. Em contrapartida, o enxugamento da tabela de *streaming* segundo o modelo *Snowflake* demonstrou poupar carga latente na memória dos injetores do Apache Druid. A integração final do Dagster como orquestrador de *Assets* de software e a fixação de manuais estritos de desenvolvimento para IA garantem que a plataforma continue expandindo suas capacidades de modelagem preditiva no setor geoespacial de modo contínuo, resiliente e padronizado.